EXCEL ENGINEERING COLLEGE
(Autonomous)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Semester
20CS402- Software Engineering
Regulations 2020

Question Bank

UNIT – V (Metrics for Process and Projects and Quality Management)

PART- A

Q.No	Questions	Marks	CO	BL
1	Classify the difference between process metrics and project metrics? • Process metrics − These characteristics can be used to improve the development and maintenance activities of the software. • Project metrics − This metrics describe the project characteristics and execution.	2	CO5	R
2	How do you measure project performance? • Units Completed. The Units Completed lends itself well to tracking tasks that are done repeatedly, • where each iteration can easily be measured. • Incremental Milestones. • Start/Finish • Cost Ratio • Experience/Opinion • Weighted or Equivalent Units	2	CO5	R
3	Interpret the objectives of software measurement? • Software measurement provides objective information to help the project manager do the Following: Communicate Effectively • Measurement provides objective information throughout the software organization. This reduces the ambiguity that often surrounds complex and constrained software projects	2	CO5	R
4	Identify the benefits of using object-oriented software metrics? Effective object-oriented designs maximize cohesion since it promotes encapsulation. The third class metrics investigates cohesion. LCOM measures the degree of similarity of methods by data input variables or attributes a) Modularity for easier troubleshooting. When working with object-oriented programming languages, you know exactly where to look when something goes wrong. b) Reuse of code through inheritance	2	CO5	R
5	List out the difficulties of project estimation. • Estimate at Completion. • Uncertainty, Assumptions and Inaccuracy. • Time Loss.	2	CO5	R

	• Misrepresentation of Project Status			
6	What is the major weakness of COCOMO? • It ignores requirements volatility (but an organization may add this as an extra adjustment factor in computing EAF). • It ignores documentation and other requirements. • It ignores customer attributes—skill, cooperation, knowledge, and responsiveness	2	CO5	R
7	Why is COCOMO important in project management? COCOMO is factual and easy to interpret. One can clearly understand how it works. - Accounts for various factors that affect cost of the project. - Works on historical data and hence is more predictable and accurate. • Basic COCOMO Model. • Intermediate COCOMO Model. • Detailed COCOMO Model.	2	CO5	R
8	How do you estimate the effort and development time in Cocomo model? • The effort is measured in Person-Months and as evident from the formula is dependent on Kilo-Lines of code. The development time is measured in months • COCOMO or Constructive Cost Estimation Model is a model that estimates the effort and time taken to complete the model based on the size of the source code. It includes 15 multiplying factors from different attributes of the project, and finally calculates time and effort using this information	2	CO5	R
9	Which decomposition technique is powerful and commonly used on large data structures? • Domain decomposition is one of the most important techniques commonly used in parallel computation. The basic idea is to divide the global domain into many subdomains and then the governing differential equations are solved in several or all sub domains simultaneously • Recursive decomposition is a powerful and flexible technique, but does not always fit well with the structure of iterative problems, and usually requires adoption of a lightweight execution framework for efficient implementation.	2	CO5	R
10	Why is FPA methodology better than LOC? • Creation of more function points can define productivity goal as opposed to LOC. e. Productivity of projects written in different languages can be measured. • Additional function point analysis benefits are the ability to assess the size of an application and to evaluate its ability to meet business or user requirements. It also serves as a productivity solution for evaluating the work completed by your team based on defined units.	2	CO5	R
11	How do function points overcome LOC problems? • Function Point Analysis circumvents the LOC issues by using	2	CO5	R

	units of function (input and output interactions, database tables, etc) to describe programs. This is a well-developed system with considerable empirical data to validate it. • Function Points are a standard measure unit that is replicable. Features may be evenly measured in Function Points, regardless of who measures them. Function Points may be used for agile projects and non agile projects,			
12	What are unadjusted use case points? • Use case points measure the size of an application. Once we know the approximate size of an application, we can derive an expected duration for the project if we also know (or can estimate) the team's rate of progress. • By multiplying each factors weight value by its influence value and summing them all together • The TF and EF can be determined. Finally the Adjusted Use Case Points (UCP) is calculated using the formula: UCP = UUCW x TCF x EF	2	CO5	R
13	Which technique is used for empirical estimation technique? • The original COCOMO model was a set of models; 3 development modes (organic, semi-detached, and embedded) and 3 levels (basic, intermediate, and advanced). • COCOMO model levels: Basic - predicted software size (lines of code) was used to estimate development effort.	2	CO5	R
14	Identify the challenges in software quality? • Last-Minute Changes to Requirements. Solution. • Inadequate information on user stories. Solution. • Inadequate Experience with Test Automation. • Inadequate collaboration between testers and developers. Solution.	2	CO5	R
15	What are steps involved in SQA planning and the various activities performed as part of planning? • Plan the review. • Review software requirement analysis. • Review the test design. • Review before release. • Review project closing.	2	CO5	R
16	Show what are the stages involved in the review of a software design? • Stage 1: Understanding project requirements. • Stage 2: Research and Analysis. • Stage 3: Design. • Stage 4: Prototyping. • Stage 5: Evaluation.	2	CO5	R
17	Point out the difference between formal review and informal review? • An informal review is often attached to a quote for a formal	2	CO5	R

	accessibility audit, providing some introductory information to a client to give them a better idea of the types of issues on their website before they commit to the formal review. An informal review can be helpful in justifying a formal review • A formal review is a type of review that follows a defined process. And, also the output of a formal review is documented. It is different from an informal review that neither requires any documentation nor, it follows a defined process			
18	How can one use software reliability measures to monitor the operational performance of software? • Mean Time to Failure (MTTF) • Mean Time to Repair (MTTR) • Mean Time Between Failure (MTBR) • Rate of occurrence of failure (ROCOF) • Probability of Failure on Demand (POFOD) • Availability (AVAIL)	2	CO5	R
19	Which are the major factors that influence software costs? • Development timelines. Time to Market has a tremendous impact on the success of your project. • Team skills. • The complexity of the product. • Prototype & design. • Architecture components. • Tools & process, methodology. • Third-party integrations.	2	CO5	R
20	Indicate the effective methods to ensure the success of SQA? • Creating an SQA Management Plan • Setting the Checkpoints • Apply software Engineering Techniques • Executing Formal Technical Reviews • Having a Multi- Testing Strategy • Enforcing Process Adherence • Controlling Change • Measure Change Impact	2	CO5	R

PART- B

Q.No	Questions	Marks	CO	BL
1	**Explain the COCOMO II Model in detail and Enumerate any 4 size oriented metrics and explain them** • Internal metrics: Internal metrics are the metrics used for measuring properties that are viewed to be of greater importance to a software developer. For example, Lines of Code (LOC) measure. • External metrics: External metrics are the metrics used for measuring properties that are viewed to be of greater importance to the user, e.g., portability, reliability, functionality, usability, etc. • Hybrid metrics: Hybrid metrics are the metrics that combine product, process, and resource metrics. For example, cost per FP where FP stands for Function Point Metric. • Project metrics: Project metrics are the metrics used by the project manager to check the project's progress. Data from the past projects are used to collect various metrics, like time and cost; these estimates are used as a base of new software. Note that as the project proceeds, the project manager will check its progress from time-to-time and will compare the effort, cost, and time with the original effort, cost and time. **COCOMO II Model** COCOMO (Constructive Cost Model) is a regression model based on LOC, i.e number of Lines of Code. It is a procedural cost estimate model for software projects and is often used as a process of reliably predicting the various parameters associated with making a project such as size, effort, cost, time, and quality • COCOMO (Constructive Cost Model) is a regression model based on LOC, i.e number of Lines of Code. • It is a procedural cost estimate model for software projects and is often used as a process of reliably predicting the various parameters associated with making a project such as size, effort, cost, time, and quality. • It was proposed by Barry Boehm in 1981 and is based on the study of 63 projects, which makes it one of the best documented models. The key parameters which define the quality of any software products, which are also an outcome of the COCOMO are primarily Effort & Schedule • Effort: Amount of labor that will be required to complete a task. It is measured in person-months units. • Schedule: Simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in the units of time such as weeks, months. Different models of COCOMO have been proposed to	16	CO5	R

predict the cost estimation at different levels, based on the amount of accuracy and correctness required.

- All of these models can be applied to a variety of projects, whose characteristics determine the value of constant to be used in subsequent calculations.
- These characteristics pertaining to different system types are mentioned below. Boehm's definition of organic, semidetached, and embedded systems:
- Organic – A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem
- Semi-detached – A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, knowledge of the various programming environment lie in between that of organic and embedded. The projects classified as Semi-Detached are comparatively less familiar and difficult to develop compared to the organic ones and require more experience and better guidance and creativity. Eg: Compilers or different Embedded Systems can be considered of Semi-
- Embedded – A software project requiring the highest level of complexity, creativity, and experience requirement fall under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models.
    1. Basic COCOMO Model
    2. Intermediate COCOMO Model
    3. Detailed COCOMO Model
- Basic Model

$$E = a(KLOC)^b$$
$$TIME = C(Effort)^d$$
$$Person\ Required = Effort/Time$$

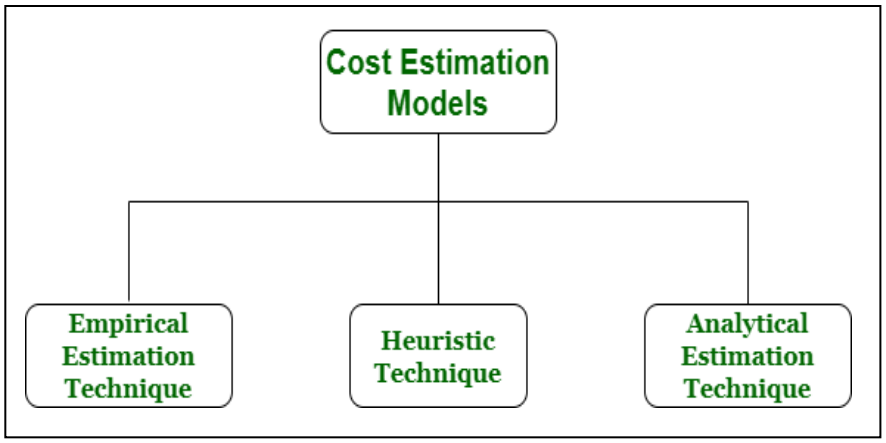
The above formula is used for the cost estimation of for the basic COCOMO model, and also is used in the subsequent models. The constant values a,b,c, and d for the Basic Model for the different categories of the system:

Software Projects	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35

		Embedded	**3.6**	**1.20**	**2.5**	**0.32**				
2	Explain in detail about Process Metrics and Project Metrics Software process and project metrics are quantitative measures that enable software engineers to gain insight into the efficiency of the software process and the projects conducted using the process framework. ● lead time, ● deployment frequency, ● mean time to restore (MTTR) ● change fail percentage **Process metrics** − These characteristics can be used to improve the development and maintenance activities of the software. Project metrics − This metrics describe the projectcharacteristics and execution. ● Sales Revenue. Tracking sales revenue helps you measure your financial performance. ... ● Customer Acquisition Costs. Customer Acquisition Costs are the expenses related toacquiring new customers. ... ● Customer Churn. ... ● Customer Engagement. ... ● Customer Satisfaction. **Process Metrics** : Process metrics are the measures of the development process that creates a body of software. A common example of a process metric is the length of time that the processof software creation tasks. Based on the assumption that the quality of the product is a direct function of the process, process metrics can be used to estimate, monitor, and improve the reliability and quality of software. ISO- 9000 certification, or "Quality Management Standards", is the generic referencefor a family of standards developed by the International Standard Organization (ISO). Often, process metrics are tools of management in its attempt to gain insight into the creation of a product that is intangible. Since the software is abstract, there is no visible, traceable artifactfrom software projects. Objectively tracking progress becomes extremely difficult. Management is interested in measuring progress and productivity and being able to makepredictions concerning both. **People Metrics** : People metrics play an important role in software project management. Theseare also called personnel metrics. Some authors view resource metrics to include personnel metrics, software metrics, and hardware metrics but most of the authors mainly view resource metrics to consist of personnel metrics only. In the present context, we also assume resource metrics to include mainly personnel metrics. People metrics quantify useful attributes of those generating the products using the available processes, methods, and tools. These metrics tell you about the attributes like turnover rates, productivity, and absenteeism.							16	CO5	U

	The goal of the people metrics is to keep staff happy, motivated, and focused on the task athand. These metrics are as : Programming Experience Metrics : Programming language experience • Development methods experience • Management experience Communication Level Metrics : • Teamwork experience • Communication hardware software level. • Personal availability. Productivity Metrics : • Size productivity • Productivity statistics • Quality vs. Productivity Team Structure Metrics • Hierarchy metrics. Team stability metrics People metrics are very helpful in assisting the appropriate allocation of resources amongst various software project activities			
3	How Use case based estimations are used to complexity of the use cases in the system? Mention the significance of use case metrics? Use case method provides a metric that assigns a software complexity metric to any set of use cases. This complexity metric may then be used to estimate the cost and level of effort associated with any use case set. The software engineering discipline has established some common measures of software complexity. Perhaps the most common measure is the McCabe essential complexity metric. This is also sometimes called cyclomatic complexity. It is a measure of the depth and quantity of routines in a piece of code. ● Cyclomatic Complexity. ● Lines of Source Code. ● Lines of Executable Code. ● Coupling/Depth of Inheritance. ● Maintainability Index. ● Cognitive Complexity. ● Halstead Volume. ● Rework Ratio. Use case points (UCP or UCPs) is a software estimation technique used to forecast the software size for software development projects. UCP is used when the ● Unified Modeling Language (UML) Rational Unified Process (RUP) Methodologies are being used for the software design and development. cyclomatic complexity of a code section is the quantitative measure of the number of linearly independent paths in it. It is a software metric used to indicate the complexity of a program. It is computed using the Control Flow Graph of the program. The nodes in the graph indicate the smallest group of	16	CO5	U

	commands of a program, and a directed edge in it connects the two nodes i.e. if second command might immediately follow the first command **Lines of Source Code** Source lines of code (SLOC), also known as lines of code (LOC), is a software metric used to measure the size of a computer program by counting the number of lines in the text of the program's source code • Lines of Executable Code. Indicates the approximate number of executable code lines or operations. This is a count of number of operations **Coupling/Depth of Inheritance:** Depth of Inheritance is similar to class coupling in that a change in a base class can affect any of its inherited classes. The higher this number, the deeper the inheritance and the higher the potential for base class modifications to result in a breaking change **Maintainability:** Index is a software metric which measures how maintainable (easy to support and change) the source code is. The maintainability index is calculated as a factored formula consisting of SLOC (Source Lines Of Code), Cyclomatic Complexity and Halstead volume. Cognitive Complexity is a measure of how difficult a unit of code is to intuitively understand. Unlike Cyclomatic Complexity, which determines how difficult your code will be to test, Cognitive Complexity tells **Halstead Volume** Halstead's software metrics is a set of measures proposed by Maurice Halstead to evaluate the complexity of a software program. These metrics are based on the number of distinct operators and operands in the program, and are used to estimate the effort required to develop and maintain the program. **Rework Ratio** The rework ratio is the relationship between the rework quantity (RQ) and produced quantity (PQ) for a work unit, product, production order, or defect type: RQ/P Rework Rate = (1 – Good Rate) x Solving Rate			
4	**i. Illustrate on empirical estimation models in detail. (8)** • Cost estimation simply means a technique that is used to find out the cost estimates. The cost estimate is the financial spend that is done on the efforts to develop and test software in Software Engineering. Cost estimation models are some mathematical algorithms or parametric equations that are used to estimate the cost of a product or a project. • Various techniques or models are available for cost estimation, also known as Cost Estimation Models as shown below :	16	CO5	U



- **Empirical Estimation Technique –**
  Empirical estimation is a technique or model in which empirically derived formulas are used for predicting the data that are a required and essential part of the software project planning step. These techniques are usually based on the data that is collected previously from a project and also based on some guesses, prior experience with the development of similar types of projects, and assumptions. It uses the size of the software to estimate the effort.
- In this technique, an educated guess of project parameters is made. Hence, these models are based on common sense. However, as there are many activities involved in empirical estimation techniques, this technique is formalized. For example Delphi technique and Expert Judgement technique.

ii. Explain SQA plan with an example. (8)

SQA Plans:

- The SQA Plan provides a road map for instituting software quality assurance. Developed by the SQA group, the plan serves as a template for SQA activities that are instituted for each software project.
- A standard for SQA plans has been recommended by the IEEE . Initial sections describe the purpose and scope of the document and indicate those software process activities that are covered by quality assurance. All documents noted in the SQA Plan are listed and all applicable standards are noted. The management section of the plan describes SQA's place in the organizational structure, SQA tasks and activities and their placement throughout the software process, and the organizational roles and responsibilities relative to product quality.

The documentation section describes (by reference) each of the work products produced as part of the software process. These include

• project documents (e.g., project plan)

	• models (e.g., ERDs, class hierarchies) • technical documents (e.g., specifications, test plans) • user documents (e.g., help files) In addition, this section defines the minimum set of work products that are acceptable to achieve high quality. The standards, practices, and conventions section lists all applicable standards and practices that are applied during the software process (e.g., document standards, coding standards, and review guidelines). In addition, all project, process, and (in some instances) product metrics that are to be collected as part of software engineering work are listed. The reviews and audits section of the plan identifies the reviews and audits to be conducted by the software engineering team, the SQA group, and the customer. It provides an overview of the approach for each review and audit. The test section references the Software Test Plan and Procedure . It also defines test record-keeping requirements. Problem reporting and corrective action defines procedures for reporting, tracking, and resolving errors and defects, and identifies the organizational responsibilities for these activities. The remainder of the SQA Plan identifies the tools and methods that support SQA activities and tasks; references software configuration management procedures for controlling change; defines a contract management approach; establishes methods for assembling, safeguarding, and maintaining all records; identifies training required to meet the needs of the plan; and defines methods for identifying, assessing, monitoring, and controlling risk.			
5	Explain in detail about Formal Technical Reviews used to improve the Quality of software. Formal Technical Review (FTR) is a software quality control activity performed by software engineers. Objectives of formal technical review (FTR): Some of these are: Useful to uncovererror in logic, function and implementation for any representation of the software. There are mainly 3 types of software reviews: • Software Peer Review: Peer review is the process of assessing the technical content andquality of the product and it is usually conducted by the author of the work product along with some other developers. ... • Software Management Review: ... • Software Audit Review: The main review types that come under the static testing are mentioned below: • Walkthrough: It is not a formal process. It is led by the authors. ...	16	CO5	R

Technical review: It is less formal review. It is led by the trained moderator but can alsobe led by a technical expert.

- Inspection:

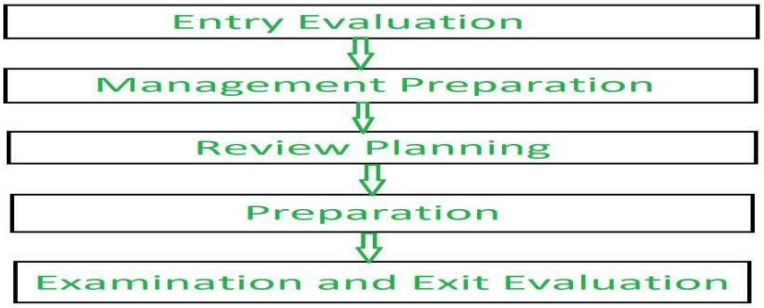
Technical Review Characteristics:

- Technical Reviews are documented and uses a defect detection process that has peersand technical specialist as part of the review process.
- The Review process doesn't involve management participation.
- It is usually led by trained moderator

who is NOT the author.Objectives for formal technical reviews

- Allow senior staff members to correct errors.
- Assess programmer productivity.
- Determining who introduced

an error into a program.uncover errors in software work products



Review guidelines :- Guidelines for the conducting of formal technical reviews should be established in advance. These guidelines must be distributed to all reviewers, agreed upon, and then followed. A review that is unregistered can often be worse than a review that does not minimum set of guidelines for FTR.

1. Review the product, not the manufacture (producer).
2. Take written notes (record purpose)
3. Limit the number of participants and insists upon advance preparation.
4. Develop a checklist for each product that is likely to be reviewed.
5. Allocate resources and time schedule for FTRs in order to maintain time schedule.
6. Conduct meaningful training for all reviewers in order to make reviews effective.
7. Reviews earlier reviews which serve as the base for the current review being conducted.
8. Set an agenda and maintain it.
9. Separate the problem areas, but do not attempt to solve every problem notes.
10. Limit debate and rebuttal.

Tools involved in requirement engineering:

- observation report
- Questionnaire (survey , poll)

	• Use cases • User stories • Requirement workshop • Mind mapping • Role playing Prototyping			
6	**Explain SQA activities with an example.** **Software Quality Assurance Activities**; **1) Creating an SQA Management Plan** Creating an SQA Management plan involves charting out a blueprint of how SQA will becarried out in the project with respect to the engineering activities while ensuring that youcorral the right talent/team. **#2) Setting the Checkpoints** The SQA team sets up periodic quality checkpoints to ensure that product development is ontrack and shaping up as expected. **#3) Support/Participate in the Software Engineering team's requirement gathering** Participate in the software engineering process to gather high-quality specifications. For gathering information, a designer may use techniques such as interviews and FAST (FunctionalAnalysis System Technique). Based on the information gathered, the software architects can prepare the project estimation using techniques such as WBS (Work Breakdown Structure), SLOC (Source Line of Codes), and FP(Functional Point) estimation. **#4) Conduct Formal Technical Reviews** An FTR is traditionally used to evaluate the quality and design of the prototype. In this process,a meeting is conducted with the technical staff to discuss the quality requirements of the software and the design quality of the prototype. This activity helps in detecting errors in the early phase of SDLC and reduces rework effort later **#5) Formulate a Multi-Testing Strategy** The multi-testing strategy employs different types of testing so that the software product can betested well from all angles to ensure better quality **#6) Enforcing Process Adherence** This activity involves coming up with processes and getting cross-functional teams to buy in onadhering to set-up systems. **This activity is a blend of two sub-activities:** • **Process Evaluation:** This ensures that the set standards for the project are followed correctly. Periodically, the process is evaluated to make sure it is working as intendedand if any adjustments need to be made. • **Process Monitoring:** Process-related metrics are collected in this step at a designated time interval and interpreted to	16	CO5	U

understand if the process is maturing as we expect it to.

#7) Controlling Change

This step is essential to ensure that the changes we make are controlled and informed. Severalmanual and automated tools are employed to make this happen.

By validating the change requests, evaluating the nature of change, and controlling the changeeffect, it is ensured that the software quality is maintained during the development and maintenance phases.

#8) Measure Change Impact

The QA team actively participates in determining the impact of changes that are brought aboutby defect fixing or infrastructure changes, etc. This step has to consider the entire system and business processes to ensure there are no unexpected side effects.

For this purpose, we use software quality metrics that allow managers and developers to observe the activities and proposed changes from the beginning till the end of SDLC and initiate corrective action wherever required.

#9) Performing SQA Audits

The SQA audit inspects the actual SDLC process followed vs. the established guidelines that were proposed. This is to validate the correctness of the planning and strategic process vs. the actual results. This activity could also expose any non-compliance issues.

#10) Maintaining Records and Reports

It is crucial to keep the necessary documentation related to SQA and share the required SQAinformation with the stakeholders. Test results, audit results, review reports, change request documentation, etc. should be kept current for analysis and historical reference.

#11) Manage Good Relations

The strength of the QA team lies in its ability to maintain harmony with various cross- functional teams. QA vs. developer conflicts should be kept at a minimum and we should lookat everyone working towards the common goal of a quality product. No one is superior or inferior to each other- we are all a team

Benefits of Software Quality Assurance (SQA):

1. SQA produces high quality software.
2. High quality application saves time and cost.
3. SQA is beneficial for better reliability.
4. SQA is beneficial in the condition of no maintenance for a long time.
5. High quality commercial software increase market share of company.
6. Improving the process of creating software.

Improves the quality of the software

(Note:*Blooms Level (R – Remember, U – Understand, AP – Apply, AZ – Analyze, E – Evaluate, C – Create)
PART A- Blooms Level : Remember, Understand, Apply
PART B- Blooms Level: Understand, Apply, Analyze, Evaluate(if possible)
Marks: 16 Marks, 8+8 Marks, 10+6 Marks)

Subject In charge (Name & Signature)	**Course Coordinator (Name & Signature)**	**HOD**	**IQAC**